

R Social Sciences Lesson

Bianca Peterson (Ph.D.) 

bianca@fathomdata.dev

Fathom Data

Attribution

This lesson is based on the [Social Science Data Carpentry lesson](#).

All Data Carpentry instructional material is made available under the Creative Commons Attribution license.

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Install required R packages

```
1 install.packages(c("tidyverse", "here"))
```

Download the data

We will download the data directly from R during the lessons. However, if you are expecting problems with the network, it may be better to download the data beforehand and store it on your machine.

The data files for the lesson can be downloaded manually here: https://github.com/datacarpentry/r-socialsci/blob/main/episodes/data/SAFI_clean.csv

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

What is R and RStudio?

- R -> both the programming language and the software that interprets the scripts
- RStudio -> a graphical front end (interface) to R, which makes using R a lot nicer

Why learn R?

- R doesn't involve lots of pointing and clicking - easy to redo analysis if you collected more data.
- R code is great for **reproducibility** - obtain same results from same dataset using same analysis.
- R is **extensible** and **interdisciplinary** - ~20,000 packages to extend its capabilities, and statistical approaches from many scientific disciplines can be combined.
- R works on **data of all shapes and sizes** - it is designed for data analysis.
- R produces **high-quality graphics** - can adjust any aspect of your graph.
- R has a **large community** - mailing lists and websites (Stack Overflow).
- R is **open-source** and **cross-platform** - Anyone can inspect the source code.
- Less chance for mistakes.

What can you do in R?

Maths

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 66 & 78 & 90 \\ 78 & 93 & 108 \\ 90 & 108 & 126 \end{bmatrix}$$

$\det(A) = 0$

$\text{eigen}(A) = c(16.12, -1.12, -1.00)$

Statistics

```
# Linear model
fit <- lm(mpg ~ wt + hp, data = mtcars)
summary(fit)
```

Coefficients:
(Intercept) 37.2859 ***
wt -5.3445 ***
hp -0.0318 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

$t.test(x, mu = 0)$
p-value = 2.2e-16

Writing custom functions

```
standardize <- function(x) {
  mu <- mean(x, na.rm = TRUE)
  sigma <- sd(x, na.rm = TRUE)
  (x - mu) / sigma
}
```

$z <- \text{standardize}(\text{iris}\$Sepal.Length)$

$\text{summary}(z)$

Min. 1st Qu. Median Mean 3rd Qu. Max.
-2.264 -0.689 -0.042 0.000 0.633 2.620

Create Reuse Share

Data analysis

```
library(dplyr)
sales %>%
  filter(year == 2024) %>%
  group_by(category) %>%
  summarise(
    revenue = sum(amount),
    n_orders = n()
  ) %>%
  arrange(desc(revenue))
```

A tibble: 5 x 3

category	revenue	n_orders
1 Electronics	2,345,000	1,236
2 Clothing	1,278,400	987
3 Home	987,600	732
4 Books	456,300	421
5 Sports	321,900	310

Visualisation

Dashboards

R is versatile.

Powerful Flexible Insightful Reproducible Community

Presentations

Academic articles

An R Workflow for Reproducible Research

Jane Doe¹, John Smith²
¹ University A, ² University B

Abstract
We present an R-based workflow that enhances reproducibility in statistical research...

1 Introduction
R provides a complete environment for data analysis and communication...

```
results <- lm(y ~ x1 + x2, data = df)
stargazer(results, type = "text")
```

	Dependent variable:
x1	0.512*** (0.162)
x2	-0.231* (0.094)
Constant	1.234*** (0.345)
Observations	100
R ²	0.642

Note: * p<0.05; ** p<0.01; *** p<0.001

Books

Version control

```
$ git status
On branch main
Changes to be committed:
  modified:   analysis.R
  new file:   figures/plot.png

$ git commit -m "Add analysis"
[main a1b2c3d] Add analysis
2 files changed, 42 insertions(+)
```

GitHub

git

Relational databases

```
library(DBI)
con <- dbConnect(
  RPostgres::Postgres(),
  dbname = "sales",
  host = "localhost"
)
df <- dbGetQuery(con,
  "SELECT * FROM orders
  WHERE year = 2024")
dbDisconnect(con)
```

Machine learning

```
library(caret)
set.seed(123)
train_ctrl <- trainControl(
  method = "cv", number = 5
)
model <- train(
  Species ~ .,
  data = iris,
  method = "rf",
  trControl = train_ctrl
)
print(model)
```

	Predicted		
	setosa	versicolour	virginica
Actual \			
setosa	50	0	0
versicolour	0	47	3
virginica	0	2	48

Accuracy: 0.97

Neural networks

```
library(keras)
model <- keras_model_sequential() %>%
  layer_dense(64, activation = "relu",
    input_shape = c(784)) %>%
  layer_dense(10, activation = "softmax")
model %>%
  compile(optimizer = "adam",
    loss = "sparse_categorical_crossentropy",
    metrics = "accuracy")
model %>% fit(x_train, y_train,
  epochs = 10, batch_size = 128)
```

Modelling

```
library(lme4)
m <- lmer(y ~ x + (1|group), data = df)
summary(m)
AIC(m)
plot(resid(m) ~ fitted(m))
```

One language. Endless possibilities.

Import Clean Transform Analyze Visualize Communicate Automate Share

A thriving global community

Objectives

- What is R and RStudio
- **RStudio layout**
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Layout of RStudio

The image shows the RStudio interface with several panels and annotations explaining their functions using a kitchen analogy.

Scripts are recipes (records) of how to do things.
Write and save your recipes here so that R knows what to cook.

The console is where the cooking happens - send recipes here (run code) to cook them.
You can cook here without using a recipe, but you'll struggle to remember exactly how to recreate the dish in the future.
It's better to use a recipe!

The environment is like the kitchen counter - you can put ingredients (data) and finished dishes (model outputs) here to use while you cook.

Files are like ingredients in your cupboard - you need to get them out on to the kitchen counter (the environment) to use them.
The files that you need can be specified in the recipe so you know exactly what you need to get out.

Packages are like tools - when you need to use a saucepan, you go out and buy one that someone has already designed and made [install.packages()].
Each time you want to use that pan, you just take it out of the cupboard [library()].

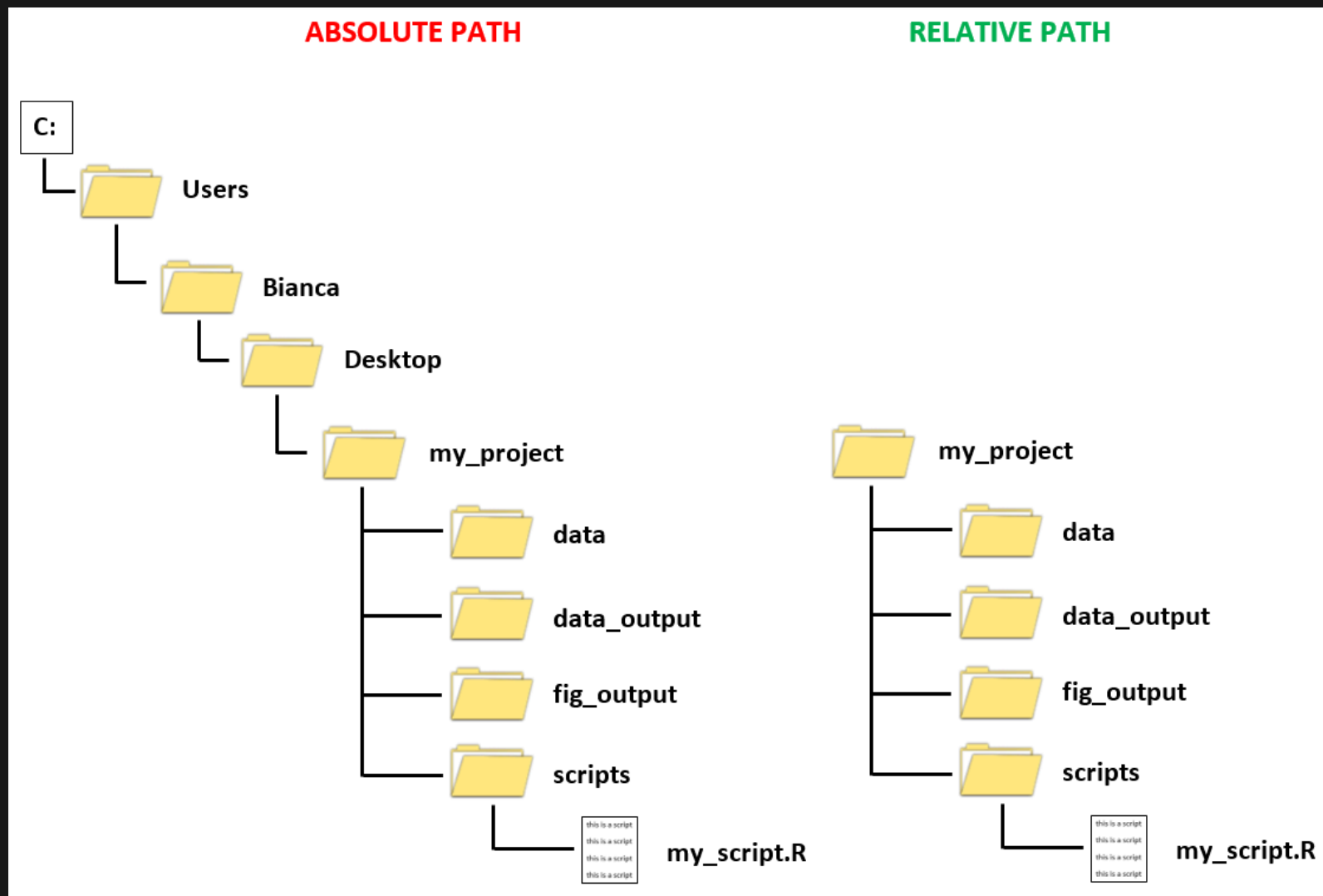
Objectives

- What is R and RStudio
- RStudio layout
- **Creating a reproducible project**
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Create a new project

- It is good practice to keep a set of related data, analyses and text in single folder called the **working directory**.
- You can easily share it with others without worrying about whether or not underlying scripts will still work.
- Only ever use relative paths and not absolute paths.

Absolute vs relative paths

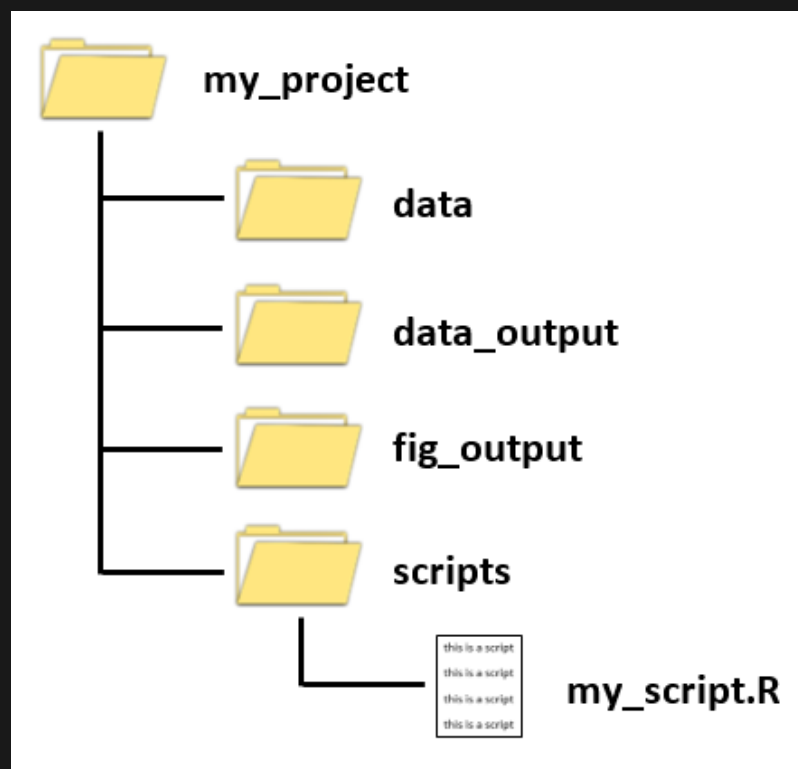


Time for...



PRACTICAL

Your working directory should look like this



Working directory = where R looks for files that you ask it to load, and where it will put any files that you ask it to save.

Interacting with R

- We write code/instructions = commands
- The computer executes/runs those commands
- Two ways to interact with R:
 - Type in the console and press **Enter** to execute -> forgotten when closed
 - Type in script editor and press **Ctrl/Cmd + Enter** -> save code and workflow
- Prompt **>** is displayed when R is ready to accept commands
- Receives commands (typing, copy-pasting or sent from script editor) -> execute -> show results -> new prompt
- If R waits for more data -> will show a **+** prompt (continuation symbol), which means you haven't finished entering a complete command
- Either enter the missing letter/value/symbol or click inside the console and press **Esc**

Time for...



PRACTICAL

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- **Object assignment**
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Time for...



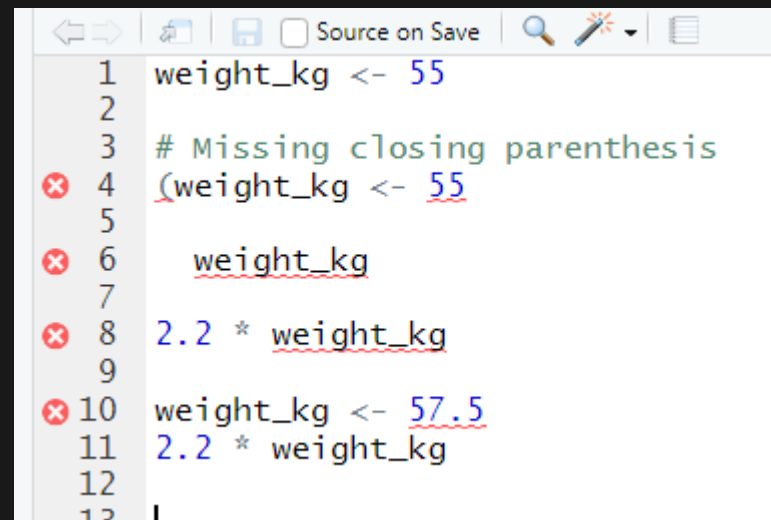
PRACTICAL

Seeking help (1/3)

- Use built-in RStudio help interface (bottom-right panel)
- Search function documentation with ? and ??
 - `?function`
 - `??task`
- `help(function)`
- Automatic code completion
 - Reminder of a function's name or arguments
 - Avoid spelling errors
- Package vignettes and cheat sheets
 - Instructions for how to use the package
 - `browseVignettes()`
 - Cheat sheets, keyboard shortcuts, and more, available in the Help menu (at the top of the RStudio window)

Seeking help (2/3)

- Finding more functions and packages
 - Help only searches installed packages
 - To search all available packages, use rdokumentation.org website
 - Google R <task>
 - Many packages also have websites with additional help, tutorials, etc.
- Dealing with error messages



A screenshot of an R script editor window. The window has a toolbar at the top with icons for undo, redo, save, and search. Below the toolbar, the script content is displayed with line numbers 1 through 13 on the left. The script contains several syntax errors, each marked with a red 'x' icon in the left margin. The errors are: line 4, a missing closing parenthesis in the assignment statement; line 6, an unterminated string literal; line 8, a missing closing parenthesis in the multiplication statement; and line 10, a missing closing parenthesis in the assignment statement. The script content is as follows:

```
1 weight_kg <- 55
2
3 # Missing closing parenthesis
4 (weight_kg <- 55
5
6   weight_kg
7
8 2.2 * weight_kg
9
10 weight_kg <- 57.5
11 2.2 * weight_kg
12
13
```

Seeking help (3/3)

- Where to ask for help:
 - Ask somebody “in the real world”
 - Stack Overflow: <http://stackoverflow.com/questions/tagged/r>
 - R-help mailing list: <https://stat.ethz.ch/mailman/listinfo/r-help>
 - Use correct vocabulary
 - Check for package-specific mailing lists
 - List of topic-specific mailing lists: <http://www.r-project.org/mail.html>
 - Generative AI
 - It can be helpful, but not always reliable or just plain wrong
 - Keep sensitive data safe!

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- **Functions and arguments**
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Structure of R expressions

```
object <- function(argument1, argument2, ...)
```

- object name = can be any word you like, but avoid dots (and no spaces!)
 - also watch out for existing function names
- `<-` = assignment operator (Shortcut: `Alt -`)
- function = name of the function followed directly by `()`
 - Example: `x <- round(3.14159)`
- arguments = specified within the `()` of the function, separated by commas
 - Example: `x <- round(3.14159, digits = 2)`

Time for...



PRACTICAL

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- **Vectors**
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Data structures vs Data types

Data structures:

- Vector
- Matrix
- Array
- Data Frame
- List
- Factor

Data types:

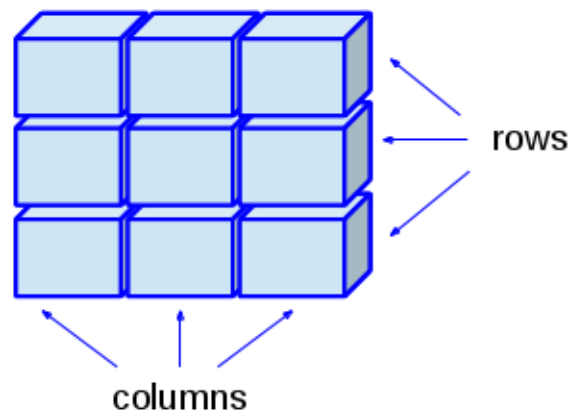
- Numeric
- Integer
- Complex
- Logical
- Character

Data structures

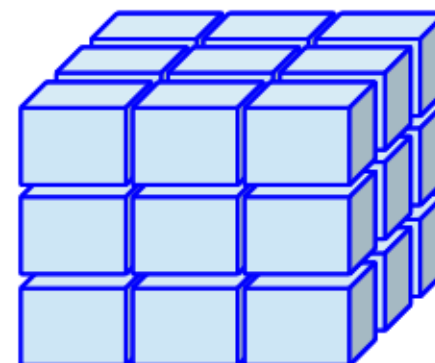
Vector



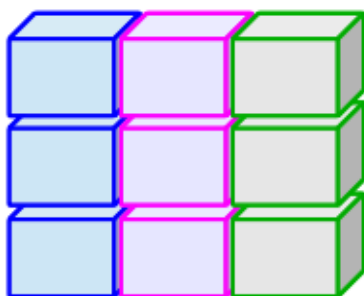
Matrix



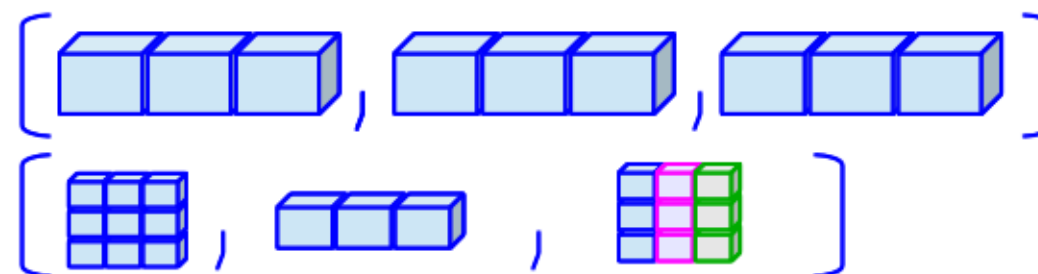
Array



Data Frame
(Table)

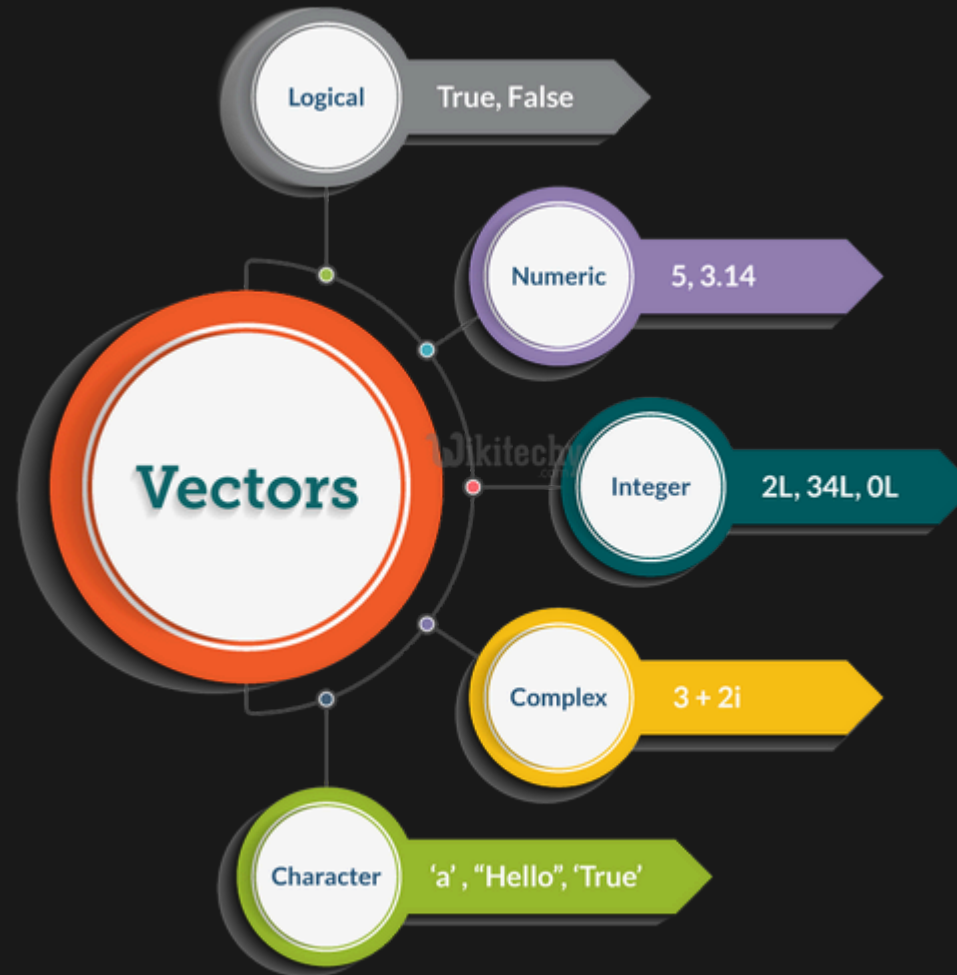


Lists



Credit: Maite Ceballos (IFCA) & Nicolas Cardiel (UCM)

Data types



Credit: [Venkatesan Prabu .J \(Wikitechy\)](#)

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Time for...



PRACTICAL

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- **Data frames**
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Importing data with `read_csv()`

Syntax	Description
<code>read_csv("data.csv", skip = 5)</code>	Skip first n lines
<code>read_csv("data.csv", comment = "#")</code>	Drop lines starting with a given character
<code>read_csv("data.csv", col_names = FALSE)</code>	No column names; label sequentially X1 to Xn
<code>read_csv("data.csv", col_names = c("name", "surname", "age", "weight"))</code>	Supply a character vector as column names
<code>read_csv("data.csv", na = ".")</code>	Specify value(s) representing missing data
<code>read_csv("data.csv", trim_ws = TRUE)</code>	Trim whitespace

Data frames

1	"S"	TRUE
7	"A"	FALSE
3	"U"	TRUE
numeric	character	logical

Tidy data

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	128042583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	128042583

observations

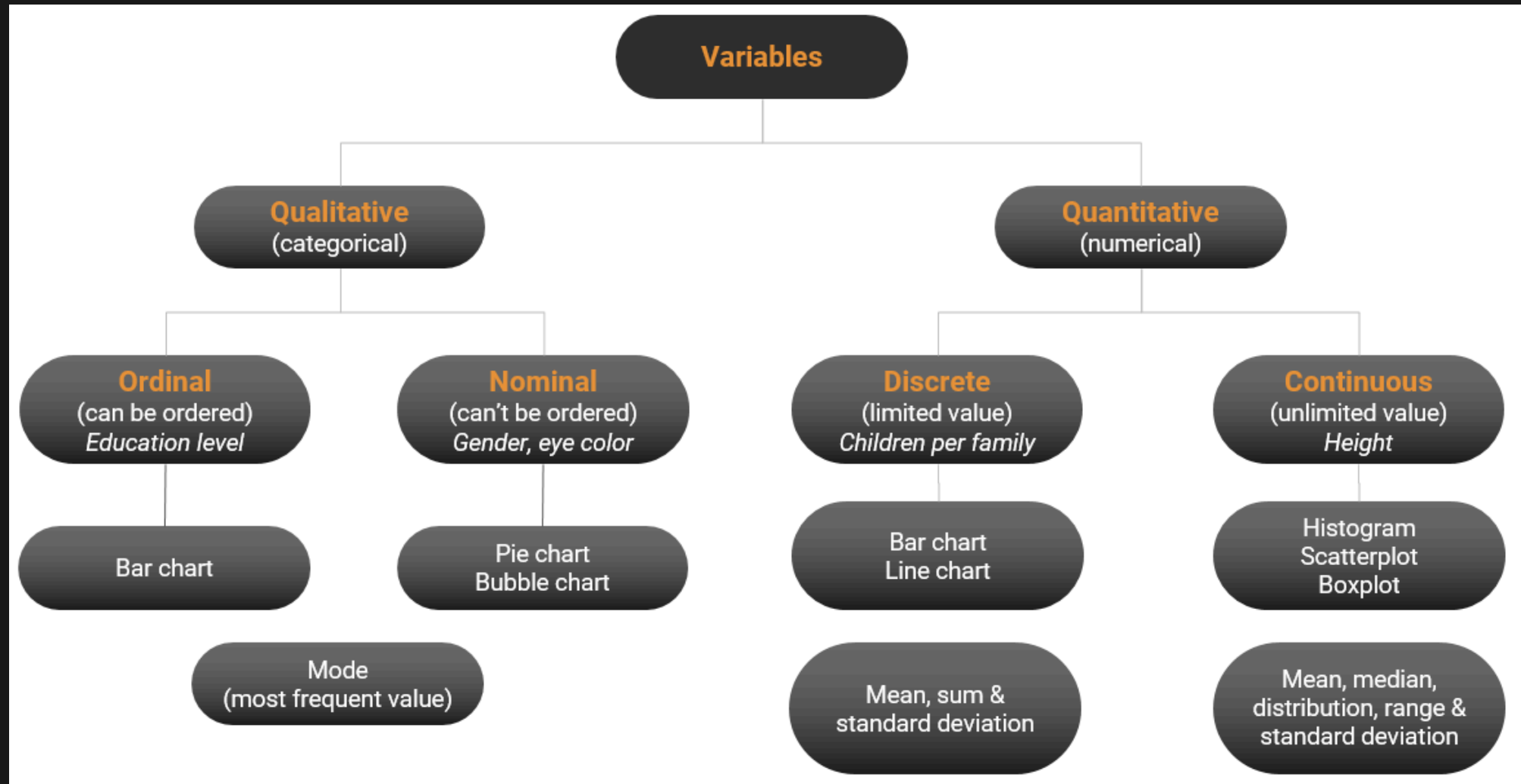
country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	128042583

values

The data

- SAFI (Studying African Farmer-Led Irrigation) is a study looking at farming and irrigation methods in Tanzania and Mozambique
- The survey data was collected through interviews conducted between November 2016 and June 2017
- The data set is stored as a comma separated value (CSV) file
- Each row contains information about a single interview respondent
- Each column represents a single variable
- A description of each variable is available [here](#)

Data types



Time for...



PRACTICAL

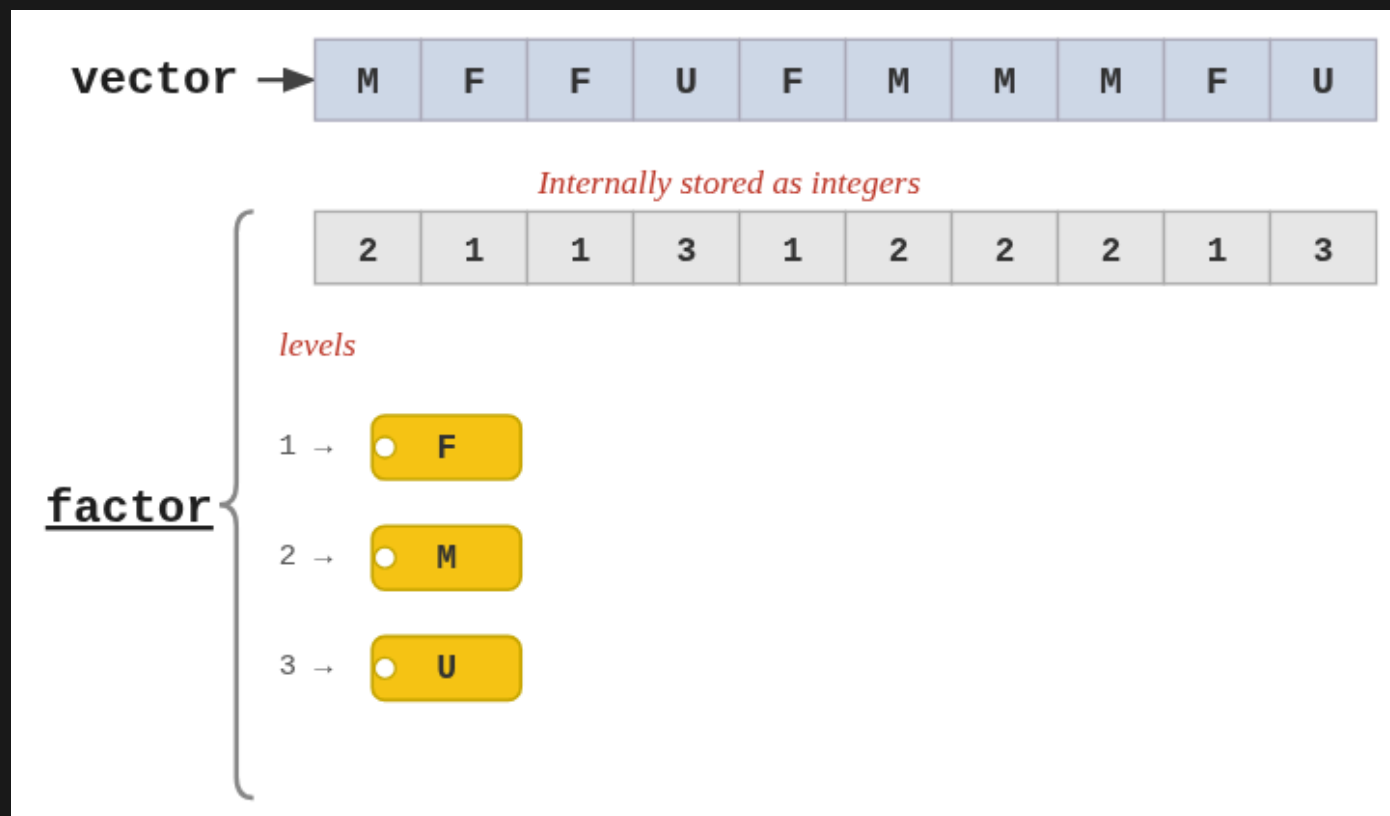
Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- **Factors**
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Factors

```
1 x <- factor(c("M", "F", "F", "U", "F", "M", "M", "M", "F", "U"))  
2 levels(x)
```

```
[1] "F" "M" "U"
```



Time for...



PRACTICAL

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- **Formatting dates**
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Formatting dates

Order of date elements	Parse function
year month day	ymd()
year day month	ydm()
month day year	mdy()
hour minute	hm()
hour minute second	hms()
year month day hour minute second	ymd_hms()

Component	Format	Example
Day of Month	%d	01
Weekday (abbreviated form)	%a	Mon
Weekday (unabbreviated form)	%A	Monday
Month (numeric form)	%m	01
Month (abbreviated character form)	%b	Jan
Month (unabbreviated character form)	%B	January
Year (numeric 2 digit form)	%y	14
Year (numeric 4 digit form)	%Y	2014

Time for...



PRACTICAL

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- **Data manipulation**
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Data manipulation using `dplyr` and `tidyr`

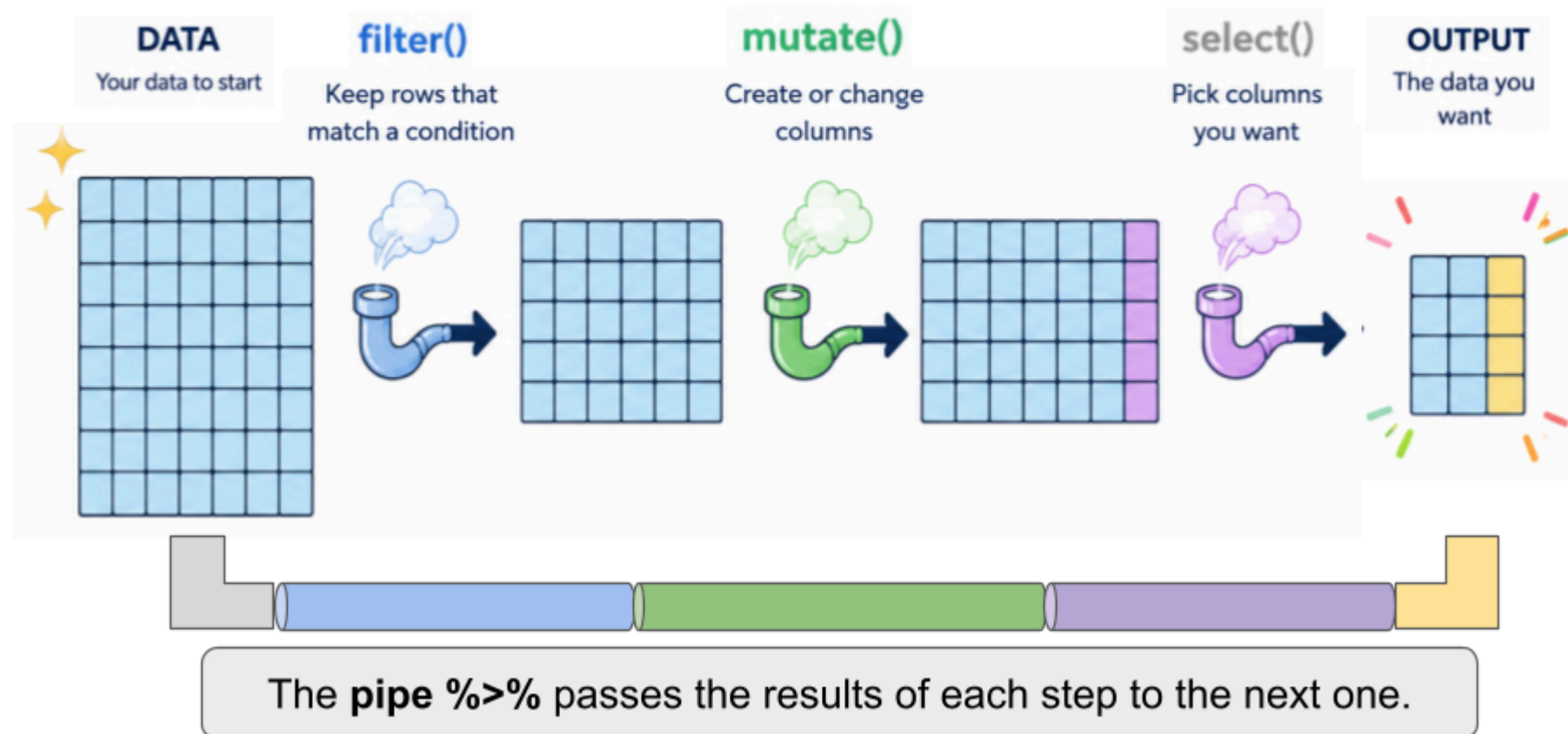
- `{dplyr}` package makes tabular data manipulation easier
- `{tidyr}` package enables you to convert between different data formats for plotting and analysis
- New functions/analyses are developed and made available via packages
- Packages = additional functions that let you do more stuff
- Install package once - `install.packages("package_name")`
- Load package with every R session when you need it - `library(package_name)`
- `{tidyverse}` = umbrella package that installs several packages

Time for...



PRACTICAL

Pipes

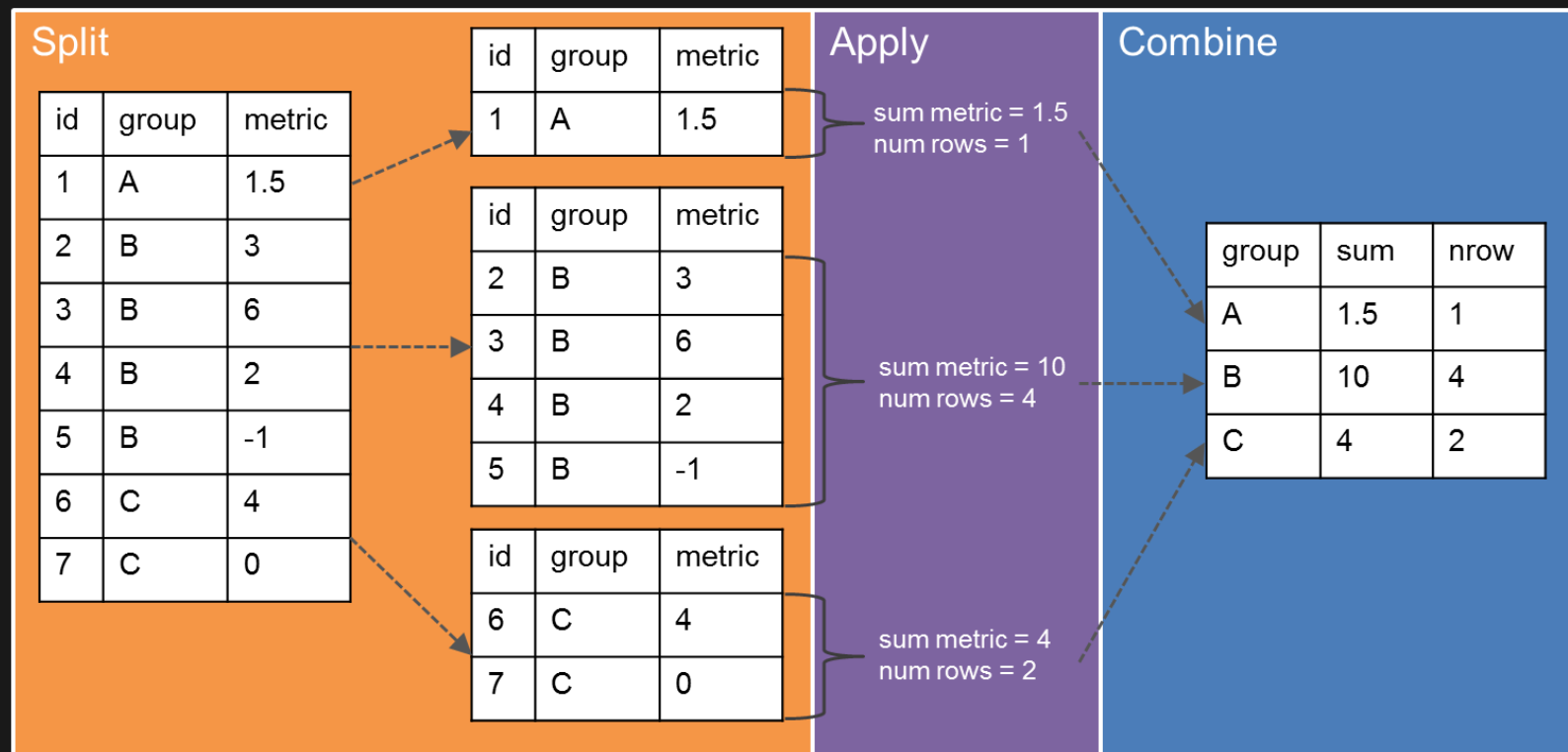


Time for...



PRACTICAL

Split-apply-combine analysis



It collapses each group into a single-row summary of that group

Credit: Modified from [Ranae Dietzel & Andee Kaplan](#)

Time for...



PRACTICAL

Reshaping your data:

separate_longer_delim

interviews %>%

```
separate_longer_delim(items_owned, delim = ",")
```

key_ID	items_owned
1	bicycle;television;solar_panel;table
2	cow_cart;bicycle;radio;cow_plough;solar_panel;solar...
3	solar_torch
4	bicycle;radio;cow_plough;solar_panel;mobile_phone
5	motorcycle;radio;cow_plough;mobile_phone
6	NA
7	motorcycle;cow_plough
8	motorcycle;bicycle;television;radio;cow_plough;solar...
9	television;solar_panel;solar_torch
10	cow_cart;motorcycle;bicycle;television;radio;cow_plo...

key_ID	items_owned
1	bicycle
1	television
1	solar_panel
1	table
2	cow_cart
2	bicycle
2	radio
2	cow_plough
2	solar_panel
2	solar_torch

Reshaping your data: `replace_na`

interviews %>%

separate_longer_delim(items_owned, delim = ";") %>%

`replace_na(list(items_owned = "no_listed_items"))`

key_ID	items_owned
1	bicycle;television;solar_panel;table
2	cow_cart;bicycle;radio;cow_plough;solar_panel;solar...
3	solar_torch
4	bicycle;radio;cow_plough;solar_panel;mobile_phone
5	motorcycle;radio;cow_plough;mobile_phone
6	NA
7	motorcycle;cow_plough
8	motorcycle;bicycle;television;radio;cow_plough;solar...
9	television;solar_panel;solar_torch
10	cow_cart;motorcycle;bicycle;television;radio;cow_plo...

key_ID	items_owned
5	motorcycle
5	radio
5	cow_plough
5	mobile_phone
6	no_listed_items
7	motorcycle
7	cow_plough
8	motorcycle
8	bicycle
8	television

Reshaping your data: **mutate**

interviews %>%

separate_longer_delim(items_owned, delim = ";") %>%

replace_na(list(items_owned = "no_listed_items")) %>%

mutate(items_owned_logical = TRUE)

key_ID	items_owned	items_owned_logical
1	bicycle	TRUE
1	television	TRUE
1	solar_panel	TRUE
1	table	TRUE
2	cow_cart	TRUE
2	bicycle	TRUE
2	radio	TRUE
2	cow_plough	TRUE
2	solar_panel	TRUE
2	solar_torch	TRUE

Reshaping your data: `if_else`

```
interviews %>%
```

```
  separate_longer_delim(items_owned, delim = ";") %>%
```

```
  replace_na(list(items_owned = "no_listed_items")) %>%
```

```
  mutate(items_owned_logical = TRUE) %>%
```

```
  group_by(key_ID) %>%
```

```
  mutate(number_items = if_else(items_owned == "no_listed_items", 0, n()))
```



key_ID	items_owned	items_owned_logical	number_items
19	bicycle	TRUE	6
19	radio	TRUE	6
19	cow_plough	TRUE	6
19	solar_panel	TRUE	6
19	solar_torch	TRUE	6
19	mobile_phone	TRUE	6
20	bicycle	TRUE	3
20	cow_plough	TRUE	3
20	solar_torch	TRUE	3
21	no_listed_items	TRUE	0
22	radio	TRUE	1

Reshaping your data: **pivot_wider**

```
interviews %>%
```

```
  separate_longer_delim(items_owned, delim = ";") %>%
```

```
  replace_na(list(items_owned = "no_listed_items")) %>%
```

```
  mutate(items_owned_logical = TRUE) %>%
```

```
  group_by(key_ID) %>%
```

```
  mutate(number_items = if_else(items_owned == "no_listed_items", 0, n())) %>%
```

```
  pivot_wider(names_from = items_owned,
```

```
    values_from = items_owned_logical,
```

```
    values_fill = list(items_owned_logical = FALSE))
```

key_ID	items_owned	items_owned_logical	number_items
1	bicycle	TRUE	4
1	television	TRUE	4
1	solar_panel	TRUE	4
1	table	TRUE	4
2	cow_cart	TRUE	8
2	bicycle	TRUE	8
2	radio	TRUE	8
2	cow_plough	TRUE	8
2	solar_panel	TRUE	8
2	solar_torch	TRUE	8

key_ID	number_items	bicycle	television	solar_panel
1	4	TRUE	TRUE	TRUE
2	8	TRUE	FALSE	TRUE
3	1	FALSE	FALSE	FALSE
4	5	TRUE	FALSE	TRUE
5	4	FALSE	FALSE	FALSE
6	0	FALSE	FALSE	FALSE
7	2	FALSE	FALSE	FALSE
8	9	TRUE	TRUE	TRUE
9	3	FALSE	TRUE	TRUE
10	9	TRUE	TRUE	TRUE

Time for...



PRACTICAL

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

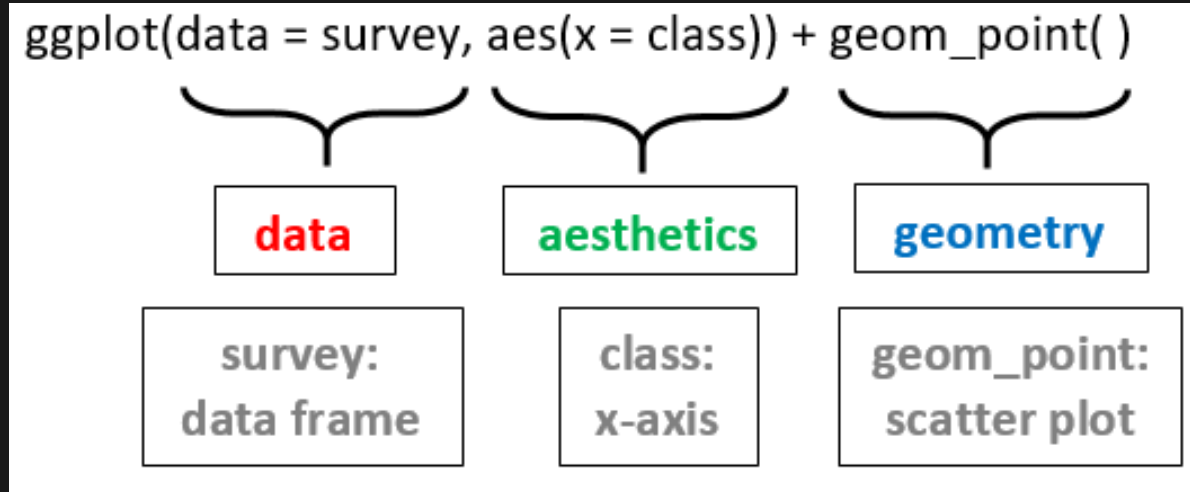
Data visualisation: grammar of graphics

Grammar of graphics = a framework which follows a layered approach to describe and construct visualisations or graphics in a structured manner

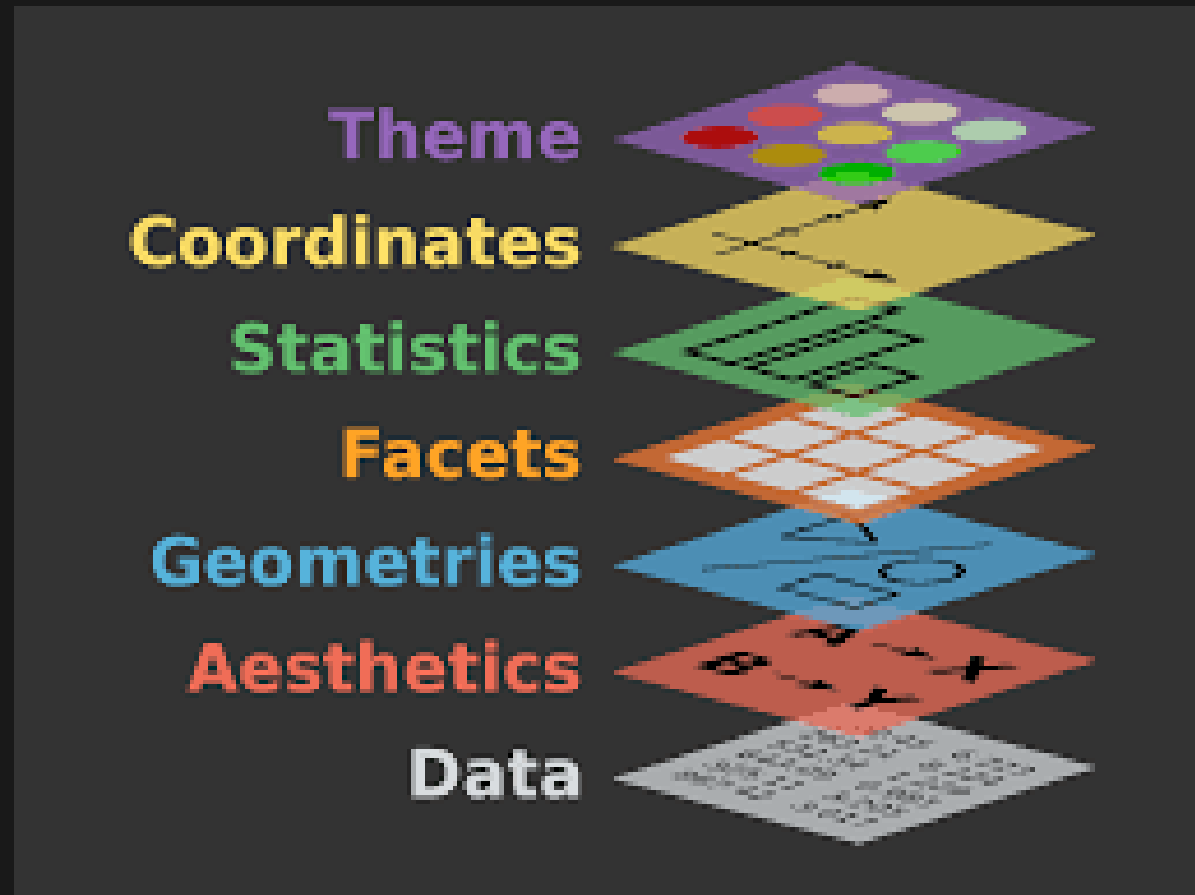
plot = **data** + **aesthetics** + **geometry**

- **<data>** = refers to a data frame (data set)
- **<aesthetics>** = indicates x and y variables & how data should be displayed in the plot (e.g. color, size, shape)
- **<geometry>** = refers to the type of graphics (bar chart, histogram, box plot, line plot, density plot, etc.)

Data visualisation: grammar of graphics



Data visualisation: grammar of graphics

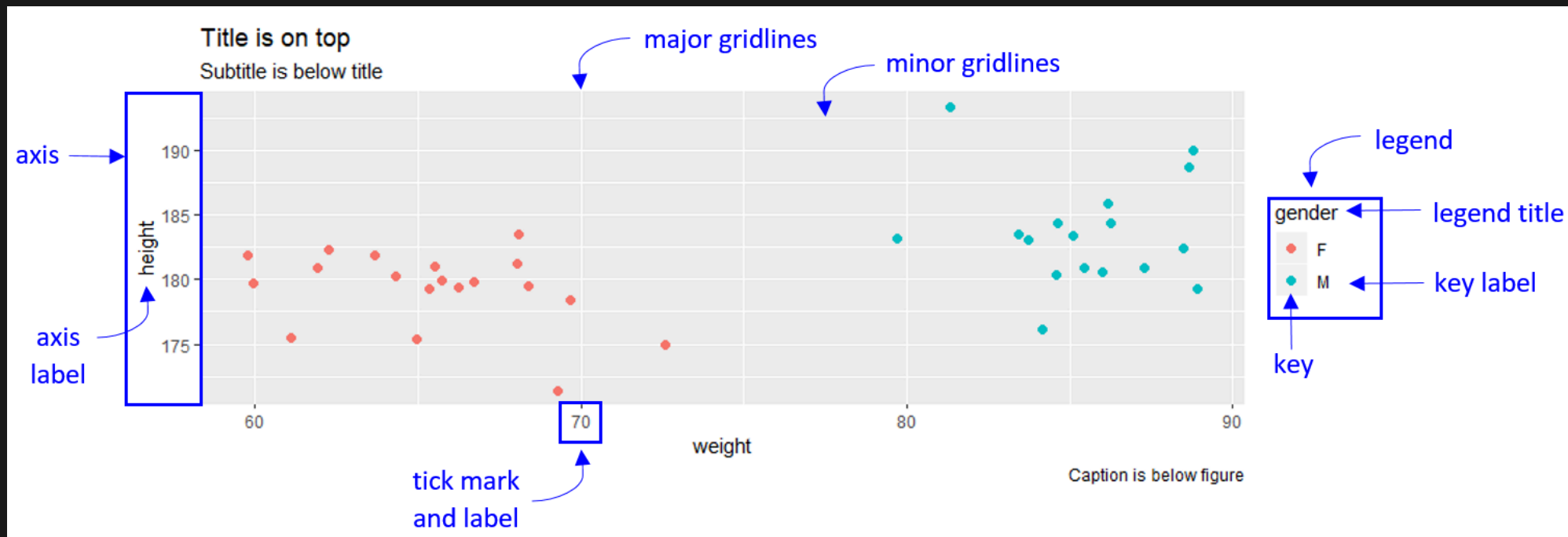


Time for...



PRACTICAL

Plot components



Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- Summary

Some other useful functions (1/5)

```
1 # Import data by browsing for file
2 my_data <- read.csv(file.choose(new = TRUE))
3
4 # Import Excel file (per sheet)
5 library(readxl)
6
7 read_excel("data/my_data.xlsx", sheet = "Sheet1")
```

Some other useful functions (2/5)

```
1 # Update a package
2 tidyverse_update()
3
4 # Helper function to use within filter():
5 between(weight, 30, 50)
6
7 # Helper functions to use within select():
8 ends_with("xyz")
9 contains("cde")
10 select(df, col5, col6, everything())
11
12 # Create and keep only the new column:
13 transmute(df, new_column = old_column*2)
14
15 # Global substitutions (i.e. find and replace):
16 gsub(pattern, replacement, data)
```

Some other useful functions (3/5)

```
1 # Remove an object from the environment
2 rm(object_name)
3
4 # Detach (i.e. "unload") a package
5 detach(tidyverse)
6
7 # Pick the top one (from arranged data)
8 dataframe %>%
9   filter(!is.na(var1)) %>%
10  arrange(var1, var2, var3) %>%
11  slice(1)
12
13 dataframe %>%
14   filter(!is.na(var1)) %>%
15   arrange(var1, var2, var3) %>%
16   top_n(1)
```

Some other useful functions (4/5)

```
1 # Divide data into categories:
2 mutate(category = case_when(weight <= 100 ~ "small",
3                               weight > 100 & weight < 200 ~ "medium",
4                               weight > 200 ~ "large"))
5
6 # Rename specific column headers:
7 rename(df, new_name = old_name)
8
9 # Top 10 heaviest animals:
10 top_n(df, 10, weight)
11
12 # Separate (i.e. split) one column into multiple:
13 separate(column, into = c("col1", "col2"))
14
15 # Bind tables on columns or rows
16 cbind(df1, df2, ...)
17 rbind(df1, df2, ...)
```

Some other useful functions (5/5)

```
1 # Mutating joins: combine variables from two data.frames
2 inner_join(x, y, by = "key") #keeps all observations in both x and y.
3 left_join(x, y, by = "key") #keeps all observations in x.
4 right_join(x, y, by = "key") #keeps all observations in y.
5 full_join(x, y, by = "key") #keeps all observations in x and y.
6
7 # Filtering joins: keep cases from the left-hand data.frame
8 semi_join(x, y) #keeps all observations in x that have a match in y.
9 anti_join(x, y) #drops all observations in x that have a match in y.
10
11 # Nesting joins: create a list column of data.frames
12 nest_join(x, y, by = "key") #keeps all rows and all columns from x
13                             #and adds a list column of tibbles
14
15 # Set operations: compare values of every variable
16 intersect(x, y) #return only observations in both x and y.
17 union(x, y)     #return unique observations in x and y.
18 setdiff(x, y)   #return observations in x that are not in y.
```

Joins

Combine Data Sets

a		b	
x1	x2	x1	x3
A	1	A	T
B	2	B	F
C	3	D	T

+

=

Mutating Joins

x1	x2	x3
A	1	T
B	2	F
C	3	NA

dplyr::left_join(a, b, by = "x1")

Join matching rows from b to a.

x1	x3	x2
A	T	1
B	F	2
D	T	NA

dplyr::right_join(a, b, by = "x1")

Join matching rows from a to b.

x1	x2	x3
A	1	T
B	2	F

dplyr::inner_join(a, b, by = "x1")

Join data. Retain only rows in both sets.

x1	x2	x3
A	1	T
B	2	F
C	3	NA
D	NA	T

dplyr::full_join(a, b, by = "x1")

Join data. Retain all values, all rows.

Filtering Joins

x1	x2
A	1
B	2

dplyr::semi_join(a, b, by = "x1")

All rows in a that have a match in b.

x1	x2
C	3

dplyr::anti_join(a, b, by = "x1")

All rows in a that do not have a match in b.

IF statements

```
1 if (condition) {  
2   # code executed when condition is TRUE  
3 } else {  
4   # code executed when condition is FALSE  
5 }  
6  
7  
8 if (this) {  
9   # do this  
10 } else if (that) {  
11   # do that  
12 } else {  
13   # do something else  
14 }
```


IF statements

```
1 temp <- 15
2
3 if (temp <= 0) {
4   "freezing"
5 } else if (temp <= 10) {
6   "cold"
7 } else if (temp <= 20) {
8   "cool"
9 } else if (temp <= 30) {
10  "warm"
11 } else {
12  "hot"
13 }
```

Writing your own function

- Functions allow you to automate common tasks instead of copy-and-pasting
- Writing a function has 3 main advantages over using copy-and-paste:
 - You can give a function an evocative name that makes your code easier to understand.
 - As requirements change, you only need to update code in one place, instead of many.
 - You eliminate the chance of making incidental mistakes when you copy and paste (i.e. updating a variable name in one place, but not in another).

Writing your own function

```
1 function_name <- function(x) {  
2   y <- read.csv(x)  
3  
4   y %>%  
5     count(variable)  
6 }  
7  
8 # Example  
9  
10 get_species <- function(x) {  
11   y <- read.csv(x)  
12  
13   y %>%  
14     count(species)  
15 }
```

Useful shortcuts

- `Ctrl + Shift + N` = New script
- `Alt + "-"` = Assignment operator `<-`
- `Ctrl + Enter` = Run selected command(s)
- `Ctrl + Shift + S` = Run the whole script
- `Ctrl + Shift + R` = Create script headers
- `Alt + Shift + K` = Keyboard shortcut quick reference
- `F1` (when function is highlighted in pop-up tooltip) = activates help window
- `Ctrl + Shift + M` = Insert pipe character `%>%`

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- **Additional resources**
- Summary

Additional Resources

- RStudio Resources
 - [R Graph Gallery](#)
 - [RStudio Primers](#)
 - [Webinars & Videos](#)
 - [Cheat Sheets](#)
 - [Educational Material for Learners or Teachers](#)
 - [A Gentle Introduction to Tidy Statistics in R](#)

Additional Resources

- Interactive learning
 - `install.packages("swirl")`
 - `library(swirl)`
 - Follow the on-screen instructions
- Books
- Introduction to R
- R FAQ
- Follow #rstats on twitter and ask questions there (also check out @RLangTips)

Objectives

- What is R and RStudio
- RStudio layout
- Creating a reproducible project
- Object assignment
- Functions and arguments
- Vectors
- Missing data
- Data frames
- Factors
- Formatting dates
- Data manipulation
- Data visualisation with ggplot2
 - Scatter plot
 - Boxplot
 - Line graph
 - Bar graph
 - Interactive plots
- Additional functions
- Additional resources
- **Summary**

Summary

- RStudio projects give you a solid workflow that will serve you well in the future:
 - Create an RStudio project for each data analysis project
 - Create a file structure that is logical and reusable - helps to automate future work
 - Keep raw data files there
 - Keep scripts there
 - Save your outputs (plots and cleaned data) there
 - Only ever use relative paths, not absolute paths